04

PANDAS

Veri Bilimci Müfredatı - Ayrıntılı Konu Anlatımı

Bu modülün amacı: Pandas ile ham tablosal veriyi güvenilir biçimde okuyabilmek, temizleyebilmek, dönüştürebilmek, birleştirebilmek, özetleyebilmek ve analize/modellemeye hazır hale getirebilmek.

Kazanım	Ne yapabileceksin?
Veri okuma	CSV/Excel/Parquet/SQL kaynaklarından veri yükleme ve tipleri kontrol etme
Seçme & filtreleme	loc/iloc, boolean mask, query ile güvenli veri seçimi
Temizleme	Eksik, duplicate, yanlış tip ve hatalı kategorileri yönetme
Aggregation	groupby, agg, transform, pivot_table ile KPI üretme
Birleştirme	merge/join/concat ile ilişkisel tabloları doğru granülerlikte birleştirme
Reshape	wide-long dönüşümleri, melt/pivot/stack/unstack
Zaman	datetime, resample, rolling ve dönem bazlı analiz
Üretim alışkanlığı	Method chaining, assign, pipe, validation ve reproducibility

Ön koşul: Python temelleri ve NumPy'de array, indexing, boolean mask ve vectorization kavramlarına aşinalık bu modülü hızlandırır.

1. Pandas nedir ve neden kullanılır?

Pandas, Python'da özellikle tablosal veriyi işlemek için kullanılan temel kütüphanedir. Veri biliminde en sık yapılan işlerin önemli bir bölümü model kurmaktan önce veriyi doğru şekle getirmektir; Pandas bu katmanın merkezindedir.

- Excel benzeri satır-sütun yapısını programlanabilir ve tekrarlanabilir hale getirir.
- SQL benzeri filtreleme, gruplama ve join işlemlerini Python içinde yapar.
- NumPy üzerine kuruludur; vektörize işlemler ve tip duyarlılığı önemlidir.
- EDA, feature engineering, deney analizi ve model öncesi hazırlıkta yoğun kullanılır.

```
import pandas as pd

df = pd.read_csv('orders.csv')
print(df.head())
print(df.shape)
print(df.dtypes)
```

Temel zihinsel model: DataFrame = indekslenmiş satırlar + isimli sütunlar + her sütuna ait dtype + eksik değer kuralları. Pandas'ta iyi olmak, bu dört unsuru sürekli takip etmektir.

2. Series ve DataFrame

Pandas'ın iki temel veri yapısı Series ve DataFrame'dir. Series tek boyutlu etiketli veri, DataFrame ise birden fazla Series'in aynı satır indeksi etrafında birleştiği iki boyutlu tablodur.

Yapı	Boyut	Örnek kullanım
Series	1D	Bir sütun, tek metrik, kategorik değişken
DataFrame	2D	Müşteri, sipariş, ürün veya özellik tablosu

```
s = pd.Series([120, 250, 310], name='revenue')

df = pd.DataFrame({
    'customer_id': [101, 102, 103],
    'city': ['Ankara', 'İstanbul', 'İzmir'],
    'revenue': [120, 250, 310]
})

print(df['revenue'])       # Series
print(df[['city','revenue']])  # DataFrame
```

Mülakat noktası: `df["col"]` Series döndürür; `df[["col"]]` ise tek sütunlu DataFrame döndürür. Bu fark shape ve sonraki işlemleri etkiler.

3. Veri okuma ve ilk profil

Veriyi okur okumaz analiz yapmak yerine önce şema ve kalite profili çıkar. İlk birkaç dakikada granülerlik, satır sayısı, sütun tipleri ve eksik değerler anlaşılmalıdır.

```
df = pd.read_csv('orders.csv')

print(df.shape)
print(df.columns.tolist())
print(df.head())
print(df.sample(5, random_state=42))
print(df.info())
print(df.describe(include='all').T)
```

Kontrol	Soru
shape	Kaç satır ve sütun var?
columns	Sütun isimleri beklenen şemaya uyuyor mu?
dtypes	Sayı, tarih, kategori tipleri doğru mu?
nunique	Kimlik ve kategori cardinality'si mantıklı mı?
isna	Eksikler nerede yoğunlaşıyor?
duplicated	Granülerliği bozan tekrarlar var mı?

```
profile = pd.DataFrame({
    'dtype': df.dtypes.astype(str),
    'missing_n': df.isna().sum(),
    'missing_pct': df.isna().mean().mul(100).round(2),
    'nunique': df.nunique(dropna=False)
})
print(profile)
```

4. Sütun ve satır seçimi: [], loc ve iloc

Pandas'ta veri seçme işlemleri açık ve niyet belirten biçimde yapılmalıdır. `loc` etiket tabanlı, `iloc` pozisyon tabanlı seçim yapar.

```
# Sütun seçimi
df['revenue']
df[['customer_id', 'revenue']]

# Etiket tabanlı
df.loc[df['revenue'] > 500, ['customer_id', 'revenue']]

# Pozisyon tabanlı
df.iloc[:10, [0, 3]]
```

Araç	Mantık	Ne zaman?
[]	Sütun/boolean mask	Basit seçimler
loc	Label-based	Koşul + sütun seçimi, güvenli assignment
iloc	Integer position	İlk/son satırlar, pozisyon bazlı seçim

Kritik alışkanlık: Filtrelenmiş bir parçaya değer yazarken chained assignment yerine `df.loc[mask, "col"] = value` kullan. Bu, belirsiz view/copy davranışını azaltır.

5. Boolean filtreleme ve query

```
high_value = df['revenue'] >= 1000
mobile = df['device'].eq('mobile')

result = df.loc[high_value & mobile]

# Birden çok değer
result = df[df['city'].isin(['Ankara','İstanbul'])]

# Aralık
result = df[df['revenue'].between(500, 2000)]

# query alternatifi
result = df.query("revenue >= 1000 and device == 'mobile'")
```

Pandas'ta `and/or/not` yerine element-wise karşılıkları olan `&`, `|`, `~` kullanılır. Birden fazla koşulu parantez içine almak okunabilirliği ve doğruluğu artırır.

```
mask = (df['revenue'] > 500) & (df['refund'] == 0)
clean_sales = df.loc[mask].copy()
```

6. Sıralama, indeks ve yeniden adlandırma

```
df = df.sort_values(['customer_id', 'order_date'])
df = df.rename(columns={'amount': 'revenue'})
df = df.reset_index(drop=True)

# index gerektiğinde
customer = df.set_index('customer_id')
```

> **Pratik kural:** Index'i "gizli primary key" gibi düşünme. Analitik kodda kimlik sütunlarını çoğu zaman açık bir kolon olarak tutmak, merge ve debugging aşamalarında daha nettir.

7. Missing values: eksik veri yönetimi

Eksik değer doğrudan "silinmesi gereken hata" değildir. Eksikliğin nedenini anlamadan drop veya imputation yapmak analiz yanlılığı üretebilir.

```
df.isna().sum()
df.isna().mean().sort_values(ascending=False)

# Eksik satırları incele
df.loc[df['age'].isna(), ['customer_id','age','segment']]

# Basit doldurma
df['city'] = df['city'].fillna('Unknown')
df['age'] = df['age'].fillna(df['age'].median())
```

Yöntem	Ne zaman?	Risk
dropna	Eksik oranı düşük ve rastlantısal ise	Örneklem küçülür, bias olabilir
fillna sabit	Unknown/Not provided anlamlıysa	Gerçek kategoriyi maskeleyebilir
median/mean	Basit sayısal baseline	Dağılımı daraltabilir
group-wise fill	Segment mantığı varsa	Leakage/yanlış grup riski
model-based	Kritik değişken ve yeterli veri varsa	Karmaşıklık ve leakage

```
df['income'] = df.groupby('segment')['income'].transform(
    lambda s: s.fillna(s.median())
)
```

8. Veri tipleri ve dönüşümler

```
df['order_date'] = pd.to_datetime(df['order_date'], errors='coerce')
df['quantity'] = pd.to_numeric(df['quantity'], errors='coerce')
df['segment'] = df['segment'].astype('category')
df['customer_id'] = df['customer_id'].astype('string')
```

Tip	Örnek	Dikkat
int/float	quantity, revenue	NaN varsa nullable Int64 gerekebilir
string	id, text	Kimlikler sayısal görünse bile string olabilir
category	segment, channel	Düşük cardinality için bellek avantajı
datetime64	order_date	Zaman işlemlerinin anahtarı
boolean	is_active	Nullable boolean gerekebilir

```
df['items'] = df['items'].astype('Int64')
df['is_active'] = df['is_active'].astype('boolean')
```

9. Duplicate ve metin temizleme

```
df.duplicated().sum()
df.duplicated(subset=['order_id']).sum()

# Kontrollü duplicate kaldırma
df = df.sort_values('updated_at').drop_duplicates(
    subset=['order_id'], keep='last'
)

# String temizleme
df['city_clean'] = (
    df['city'].astype('string')
      .str.strip()
      .str.lower()
      .str.replace(r'\s+', ' ', regex=True)
)
```

> **Granülerlik sorusu:** Bir satır neyi temsil ediyor? Sipariş mi, sipariş kalemi mi, müşteri-gün mü? Duplicate kontrolünün doğru anahtarı bu sorunun cevabına bağlıdır.

10. GroupBy: split - apply - combine

`groupby`, Pandas'ın en önemli analiz araçlarından biridir: veriyi gruplara ayırır, her gruba fonksiyon uygular ve sonucu birleştirir.

```
summary = (
    df.groupby('channel', as_index=False)
      .agg(
          orders=('order_id', 'nunique'),
          customers=('customer_id', 'nunique'),
          revenue=('revenue', 'sum'),
          avg_order_value=('revenue', 'mean')
      )
)
```

Fonksiyon	Amaç
agg	Her grup için özet satır üretir
transform	Orijinal satır sayısını koruyarak grup metriği üretir
filter	Grup koşuluna göre tüm grubu tutar/siler
apply	Esnek ama çoğu zaman daha yavaş; gerekmedikçe kaçın

```
# Her siparişin müşteri toplamı içindeki payı
df['customer_total'] = df.groupby('customer_id')['revenue'].transform('sum')
df['share_of_customer'] = df['revenue'] / df['customer_total']
```

Senior alışkanlığı: `groupby.apply` yazmadan önce aynı işin `agg`, `transform`, `rank`, `cum*` veya vektörize bir yöntemle yapılıp yapılamayacağını kontrol et.

11. Merge / Join: tabloları birleştirme

Merge işlemleri veri biliminde en fazla sessiz hata üreten alanlardan biridir. Anahtar eşsizliği ve join cardinality kontrol edilmezse satır sayısı katlanabilir ve KPI'lar bozulabilir.

Join	Ne olur?
inner	Yalnızca iki tabloda da eşleşen anahtarlar
left	Sol tablonun tüm satırları + sağdan eşleşenler
right	Sağ tablonun tüm satırları + soldan eşleşenler
outer	Her iki tablodaki tüm anahtarlar

```
orders = orders.merge(
    customers[['customer_id','segment']],
    on='customer_id',
    how='left',
    validate='many_to_one',
    indicator=True
)

print(orders['_merge'].value_counts())
```

Mutlaka kullan: `validate="one_to_one"`, `"one_to_many"`, `"many_to_one"` veya `"many_to_many"` ile beklenen join cardinality'sini kodun içine yaz. Beklenti bozulursa Pandas hata versin.

```
before = len(orders)
merged = orders.merge(customers, on='customer_id', how='left', validate='many_to_one')
assert len(merged) == before
```

12. Concat: aynı şemadaki verileri üst üste veya yan yana ekleme

```
all_months = pd.concat(
    [jan, feb, mar],
    axis=0,
    ignore_index=True
)

# Şema kontrolü
assert set(jan.columns) == set(feb.columns) == set(mar.columns)
```

`merge` ilişki anahtarlarına göre eşleştirir; `concat` ise objeleri eksen boyunca bir araya getirir. Bu ikisini zihinsel olarak ayırmak önemlidir.

13. Pivot, pivot_table, melt ve reshape

```
pivot = pd.pivot_table(
    df,
    index='month',
    columns='channel',
    values='revenue',
    aggfunc='sum',
    fill_value=0
)

long = wide.melt(
    id_vars=['customer_id'],
    var_name='month',
    value_name='revenue'
)
```

Araç	Kullanım
pivot	Anahtar kombinasyonu tekilse şekil değiştirme
pivot_table	Tekrar varsa aggregation ile pivot
melt	Wide -> long
stack/unstack	MultiIndex eksenlerini taşıma

Analitik prensip: Grafik, istatistik ve ML pipeline'larının çoğu tidy/long form veride daha rahattır: her satır bir observation, her sütun bir variable.

14. Datetime ve zaman bazlı analiz

```
df['order_date'] = pd.to_datetime(df['order_date'])
df['year'] = df['order_date'].dt.year
df['month'] = df['order_date'].dt.to_period('M')
df['weekday'] = df['order_date'].dt.day_name()
df['hour'] = df['order_date'].dt.hour

daily = (
    df.set_index('order_date')
      .resample('D')['revenue']
      .sum()
)
daily_7d = daily.rolling(7, min_periods=1).mean()
monthly = daily.resample('MS').sum()
```

Leakage uyarısı: Tahmin problemlerinde feature üretirken işlem anından sonra oluşan bilgiyi geçmişe taşımamak gerekir. Rolling/expanding işlemlerde `shift()` ihtiyacını sürekli sorgula.

15. Shift, diff, rank ve cumulative işlemler

```
df = df.sort_values(['customer_id','order_date'])

df['prev_revenue'] = df.groupby('customer_id')['revenue'].shift(1)
df['change'] = df['revenue'] - df['prev_revenue']
df['order_no'] = df.groupby('customer_id').cumcount() + 1
df['cum_revenue'] = df.groupby('customer_id')['revenue'].cumsum()
df['rank_in_customer'] = df.groupby('customer_id')['revenue'].rank(ascending=False)
```

Bu fonksiyonlar, özellikle müşteri yolculuğu, zaman serisi feature engineering ve cohort analizlerinde güçlüdür.

16. map, replace, apply ve vektörizasyon

```
segment_map = {'A':'Premium', 'B':'Standard', 'C':'Basic'}
df['segment_name'] = df['segment'].map(segment_map)

df['status'] = df['status'].replace({'complete':'completed'})
```

`apply` esnektir fakat Python seviyesinde satır/sütun dolaşımı performansı düşürebilir. Öncelik vektörize string/datetime/numpy/pandas fonksiyonları olmalıdır.

```
# Kaçınılması gereken örnek
df['net'] = df.apply(lambda r: r['revenue'] - r['cost'], axis=1)

# Vektörize
df['net'] = df['revenue'] - df['cost']
```

Tercih sırası	Örnek
1. Vektörize ifade	df[a] / df[b]
2. Pandas/NumPy built-in	where, clip, str.*, dt.*
3. map/replace	Basit sözlük dönüşümü
4. apply	Gerekliyse
5. iterrows	Nadir; çoğunlukla kaçın

17. Method chaining, assign ve pipe

Uzun notebook'larda ara değişken patlaması yerine kontrollü method chaining, veri dönüşümünün akışını görünür kılar.

```
def add_net_revenue(x):
    return x.assign(net_revenue=x['revenue'] - x['refund_amount'])

result = (
    df.loc[df['status'].eq('completed')]
      .assign(order_date=lambda x: pd.to_datetime(x['order_date']))
      .pipe(add_net_revenue)
      .groupby('channel', as_index=False)
      .agg(revenue=('net_revenue','sum'))
      .sort_values('revenue', ascending=False)
)
```

Okunabilirlik kuralı: Chaining kısa ve tek amaçlı kaldığında değerlidir. 20 adımlık zincir yerine mantıksal adımları fonksiyonlara ayır ve `pipe` ile bağla.

18. Veri doğrulama ve performans alışkanlıkları

- Her kritik dönüşümden sonra satır sayısı ve anahtar eşsizliği için assertion kullan.
- Büyük veride sadece gereken sütunları oku (`usecols`) ve uygun dtype belirt.
- CSV yerine analitik iş akışlarında Parquet gibi kolon bazlı formatlar daha verimli olabilir.
- Önce filtrele, sonra pahalı groupby/merge işlemleri yap.
- Gereksiz `.copy()` bellek tüketir; ama bağımsız bir parça üzerinde güvenli dönüşüm gerekiyorsa bilinçli kullan.
- `category` düşük cardinality stringlerde belleği azaltabilir.
- Pandas tek makine belleğine bağlıdır; veri RAM'i aşıyorsa SQL/Spark/Polars/DuckDB gibi alternatifleri değerlendirmek gerekir.

```
required = {'order_id','customer_id','revenue','order_date'}
assert required.issubset(df.columns)
assert df['order_id'].is_unique
assert df['revenue'].ge(0).all()
assert df['order_date'].notna().all()
```

19. Uçtan uca vaka: E-Commerce KPI analizi

Senaryo: Bir e-ticaret şirketi son üç ayda gelirin dalgalandığını görüyor. Amaç, sipariş ve müşteri tablolarını kullanarak kanal/ay kırılımında gelir, sipariş, müşteri ve AOV üretmek; ayrıca iade etkisini ayırmak.

orders	customers
order_id, customer_id, order_date, channel, revenue, refund_amount	customer_id, signup_date, country, segment

```
orders['order_date'] = pd.to_datetime(orders['order_date'])
orders['month'] = orders['order_date'].dt.to_period('M').astype(str)
orders['net_revenue'] = orders['revenue'] - orders['refund_amount'].fillna(0)

orders = orders.merge(
    customers[['customer_id','segment','country']],
    on='customer_id', how='left', validate='many_to_one'
)

kpi = (
    orders.groupby(['month','channel'], as_index=False)
          .agg(
              orders=('order_id','nunique'),
              customers=('customer_id','nunique'),
              gross_revenue=('revenue','sum'),
              net_revenue=('net_revenue','sum')
          )
          .assign(
              aov=lambda x: x['net_revenue'] / x['orders'],
              revenue_per_customer=lambda x: x['net_revenue'] / x['customers']
          )
)
```

Analiz soruları: Gelir düşüşü trafik/sipariş sayısından mı, AOV'dan mı, iadelerden mi geliyor? Etki tüm kanallarda mı yoksa belirli bir segmentte mi? Yeni ve mevcut müşteriler farklı mı davranıyor?

20. Pandas ile feature engineering örneği

Churn modeline müşteri seviyesinde özellik tablosu hazırlayalım. Burada granülerlik "müşteri"dir; sipariş tablosu müşteri seviyesine aggregate edilir.

```
snapshot = pd.Timestamp('2026-09-30')

features = (
    orders.loc[orders['order_date'] <= snapshot]
          .groupby('customer_id', as_index=False)
          .agg(
              order_count=('order_id','nunique'),
              total_revenue=('net_revenue','sum'),
              avg_order_value=('net_revenue','mean'),
              last_order_date=('order_date','max'),
              first_order_date=('order_date','min')
          )
          .assign(
              recency_days=lambda x: (snapshot - x['last_order_date']).dt.days,
              tenure_days=lambda x: (snapshot - x['first_order_date']).dt.days
          )
)
```

Leakage kontrolü: Snapshot tarihinden sonra oluşan siparişler feature tablosuna giremez. Label penceresi ile feature penceresini zaman ekseninde açıkça ayır.

21. Pandas - SQL zihinsel eşleştirme

SQL	Pandas
SELECT col1, col2	df[['col1','col2']]
WHERE condition	df.loc[condition]
GROUP BY	df.groupby(...)
SUM/AVG/COUNT	agg(...)
ORDER BY	sort_values(...)
JOIN	merge(...)
CASE WHEN	np.where / np.select / Series.where
DISTINCT	drop_duplicates / nunique
WINDOW	groupby + transform/rank/shift/cumsum

Veri bilimcinin iki dili birlikte düşünmesi değerlidir: büyük aggregations veritabanında, esnek analiz/feature engineering ise gerektiğinde Pandas'ta yapılabilir.

22. Sık yapılan hatalar

Hata	Neden tehlikeli?	Doğru yaklaşım
Merge sonrası satır patlaması	KPI iki/üç katına çıkabilir	validate + row-count assert
Yanlış granülerlik	Müşteri metriği sipariş kaleminde hesaplanır	Satırın temsil ettiği birimi yaz
Chained assignment	Atama belirsiz olabilir	loc ile açık assignment

Tarihleri string bırakmak	Sıralama/resample yanlışlaşır	to_datetime
Mean ile her NaN doldurmak	Dağılım ve ilişki bozulur	Eksikliği önce analiz et
apply(axis=1) alışkanlığı	Yavaş ve okunması zor	Vektörize çözüm ara
Leakage	Offline skor yapay olarak yükselir	Snapshot ve bilgi zamanı kontrolü
Index'e fazla güvenmek	Merge/debug zorlaşır	Anahtarları açık sütunda tut

23. Mülakat soruları

#	Soru	Beklenen yön
1	`loc` ve `iloc` farkı nedir?	loc label/koşul; iloc integer position.
2	`groupby().agg()` ile `transform()` farkı?	agg satır sayısını azaltır; transform orijinal index uzunluğunu korur.
3	Merge sonrası satır sayısı neden artar?	Anahtarların bir/birden çok eşleşmesi; many-to-many patlaması.
4	`pivot` ve `pivot_table` farkı?	pivot tekil kombinasyon ister; pivot_table aggregation yapabilir.
5	`apply(axis=1)` neden kaçınılabilir?	Python seviyesinde satır dolaşır, vektörize çözümler daha hızlıdır.
6	Duplicate nasıl tanımlanır?	İş granülerliğini temsil eden anahtara göre.
7	Eksik değerleri nasıl ele alırsın?	Önce kaynağı/örüntüyü analiz eder; sonra silme, sabit/grup/model imputasyonu seçerim.
8	`merge(validate=...)` ne işe yarar?	Beklenen join cardinality bozulursa hata verir.
9	Time-based feature'da leakage nasıl oluşur?	Tahmin anından sonraki bilgi geçmiş satırın feature'ına girdiğinde.
10	Pandas ne zaman uygun değildir?	Veri tek makine belleğini/CPU'sunu aşıyorsa distributed/SQL tabanlı çözümler daha uygun olabilir.

24. Alıştırmalar

1. `orders` tablosunda completed siparişlerin aylık net revenue ve AOV'sini hesapla.
2. Müşteri başına son 90 gündeki sipariş sayısını ve geliri üret.
3. `customers` ile `orders` tablolarını many-to-one doğrulamasıyla birleştir; eşleşmeyen müşteri oranını raporla.
4. Her kanal için revenue dağılımının %25, %50, %75 quantile değerlerini hesapla.
5. Bir ürün tablosunda duplicate `product_id` varsa son güncellenen kaydı tut.
6. Wide aylık gelir tablosunu `melt` ile long forma dönüştür.
7. Her müşterinin siparişlerini tarihe göre sırala ve önceki siparişten geçen gün sayısını üret.
8. İade oranı %10'dan yüksek olan kategorileri bul.
9. Her müşterinin toplam gelir içindeki sipariş payını `transform` ile hesapla.
10. Bir churn feature table için snapshot sonrası bilgiyi dışlayan veri hazırlama fonksiyonu yaz.

25. Pandas çalışma checklist'i

Aşama	Kontrol
Okuma	Şema, dtypes, tarih parse, usecols
Profil	shape, nunique, missing, duplicate
Granülerlik	Bir satır neyi temsil ediyor? primary key ne?
Temizleme	Eksik/hatalı kategori/outlier mantığı
Dönüşüm	Vektörize mi? dtype korunuyor mu?
Merge	Anahtar eşsizliği, validate, row count
Aggregation	Payda doğru mu? unique sayım gerekli mi?
Zaman	Timezone, snapshot, leakage, resample frekansı
Doğrulama	assertion, expected ranges, null checks
Çıktı	Reproducible pipeline + anlaşılır kolon isimleri

Modül çıktısı: Bu bölüm bittikten sonra bir CSV/SQL çıktısını alıp veri kalitesini kontrol edebilmeli, temizleyebilmeli, tabloları güvenli biçimde birleştirebilmeli, KPI/feature tablosu üretebilmeli ve yaptığın dönüşümlerin neden doğru olduğunu açıklayabilmelisin.

26. Mini proje - E-Commerce Customer 360

Amaç: orders, order_items, customers ve products tablolarını kullanarak müşteri seviyesinde analitik bir "Customer 360" tablosu üret.

Çıktı feature	Tanım
order_count	Tekil sipariş sayısı
total_revenue	Net toplam gelir
aov	Sipariş başına ortalama gelir
recency_days	Son siparişten snapshot'a gün
tenure_days	İlk siparişten snapshot'a gün
refund_rate	İadeli sipariş / sipariş
favorite_category	En fazla gelir üretilen kategori
active_months	Sipariş verilen tekil ay sayısı

- Her merge için cardinality beklentisini yaz ve validate kullan.
- Ara tablolarda satır sayısı / unique key assert'leri ekle.
- Snapshot tarihi tanımla ve future leakage'i engelle.
- Final tabloyu Parquet olarak kaydet.
- README'de veri granülerliği ve feature tanımlarını açıkla.